

# TASK3 DOCUMENTATION

## Project Title:

### SQL Data Analysis Using SQLite3 on Dataset for Data Analytics

## Abstract

Structured Query Language (SQL) is widely used for managing and analyzing data stored in databases. This project focuses on performing SQL Data Analysis on the "Dataset for Data Analytics" using SQLite3 in Jupyter Notebook. The cleaned dataset was imported into a SQLite database, and various SQL queries were executed to retrieve, filter, group, and summarize data. The project demonstrates how SQL can be used to transform raw data into meaningful information and support data-driven decision-making.

## 1.Introduction

Databases are essential for storing and managing large volumes of data. SQL provides an efficient way to retrieve, manipulate, and analyze data stored in databases. Through SQL queries, users can extract specific information, calculate metrics, identify trends, and generate reports.

This project focuses on performing SQL Data Analysis using SQLite3 within Jupyter Notebook. The cleaned dataset was imported into a SQLite database, and various SQL operations were performed to analyze the data. The project highlights the practical application of SQL in data analytics and business reporting.

## 2. Objectives

- To create a SQL database for storing the dataset.
- To import the cleaned dataset into a database table.
- To retrieve data using SQL queries.
- To filter records using SQL conditions.
- To group records using SQL aggregation techniques.
- To calculate summary statistics using aggregate functions.
- To derive meaningful insights from the dataset.

## 3. Problem Statement

Organizations generate and store large amounts of data daily. Analyzing this data manually is time-consuming and inefficient. SQL provides a powerful mechanism for querying and summarizing data efficiently. The objective of this project is to use SQLite3 and SQL queries to retrieve, organize, and analyze data stored in a database and generate valuable insights for decision-making.

## 4. Dataset Description

**Dataset Name:** Cleaned Dataset for Data Analytics

The dataset used in this project was originally obtained from a previous dataset. It was cleaned and preprocessed to improve data quality, and the cleaned data was then exported to a new Excel file. This

exported dataset was imported into a SQLite database and analyzed using SQL queries to generate meaningful insights.

## Dataset Information

- Dataset Name: Cleaned Dataset for Data Analytics
- File Format: Excel (.xlsx)
- Database Used: SQLite3
- Domain: Data Analytics
- Purpose: SQL Data Analysis

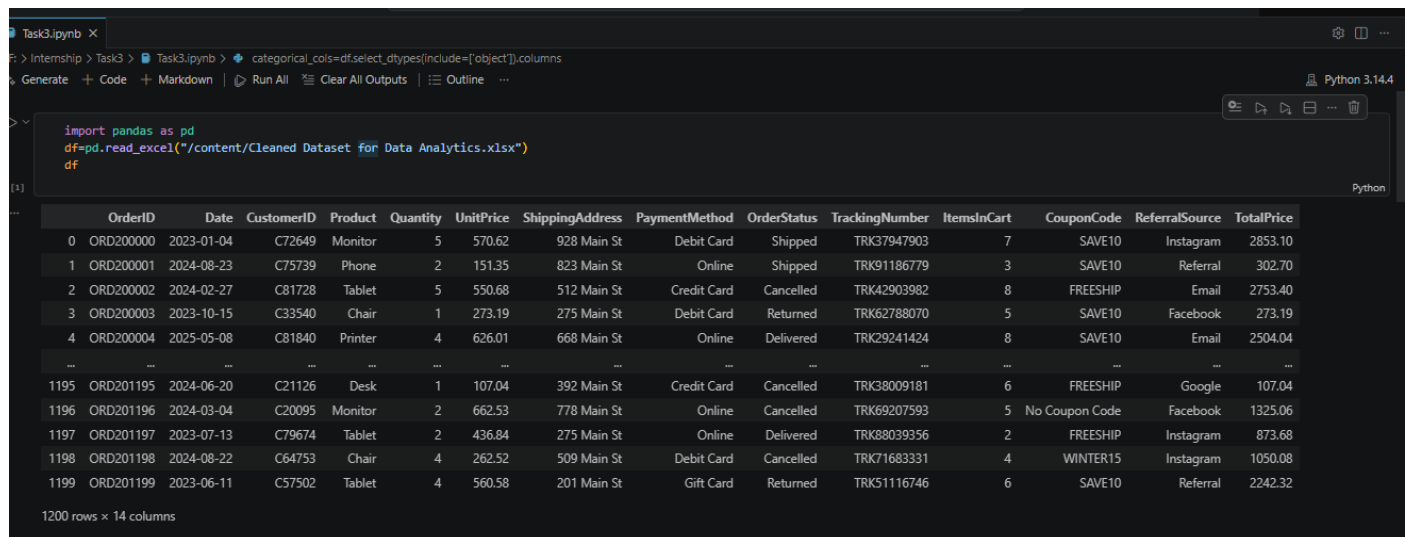
## 5. Tools and Technologies Used

- SQLite3
- SQL
- Python
- Pandas
- Jupyter Notebook
- Microsoft Excel

## 6. Methodology

### Step 1: Dataset Loading

The cleaned dataset was loaded into Jupyter Notebook using Pandas.



The screenshot shows a Jupyter Notebook interface with a code cell containing the following Python code:

```
import pandas as pd
df=pd.read_excel("/content/Cleaned Dataset for Data Analytics.xlsx")
df
```

Below the code cell, the first 12 rows of the dataset are displayed as a table. The table has 14 columns: OrderID, Date, CustomerID, Product, Quantity, UnitPrice, ShippingAddress, PaymentMethod, OrderStatus, TrackingNumber, ItemsInCart, CouponCode, ReferralSource, and TotalPrice. The data includes various products like Monitor, Phone, Tablet, Chair, Printer, and Desk, with their respective quantities, prices, shipping addresses, payment methods, order statuses, tracking numbers, items in cart, coupon codes, referral sources, and total prices.

|      | OrderID   | Date       | CustomerID | Product | Quantity | UnitPrice | ShippingAddress | PaymentMethod | OrderStatus | TrackingNumber | ItemsInCart | CouponCode     | ReferralSource | TotalPrice |
|------|-----------|------------|------------|---------|----------|-----------|-----------------|---------------|-------------|----------------|-------------|----------------|----------------|------------|
| 0    | ORD200000 | 2023-01-04 | C72649     | Monitor | 5        | 570.62    | 928 Main St     | Debit Card    | Shipped     | TRK37947903    | 7           | SAVE10         | Instagram      | 2853.10    |
| 1    | ORD200001 | 2024-08-23 | C75739     | Phone   | 2        | 151.35    | 823 Main St     | Online        | Shipped     | TRK91186779    | 3           | SAVE10         | Referral       | 302.70     |
| 2    | ORD200002 | 2024-02-27 | C81728     | Tablet  | 5        | 550.68    | 512 Main St     | Credit Card   | Cancelled   | TRK42903982    | 8           | FREESHIP       | Email          | 2753.40    |
| 3    | ORD200003 | 2023-10-15 | C33540     | Chair   | 1        | 273.19    | 275 Main St     | Debit Card    | Returned    | TRK62788070    | 5           | SAVE10         | Facebook       | 273.19     |
| 4    | ORD200004 | 2025-05-08 | C81840     | Printer | 4        | 626.01    | 668 Main St     | Online        | Delivered   | TRK29241424    | 8           | SAVE10         | Email          | 2504.04    |
| ...  | ...       | ...        | ...        | ...     | ...      | ...       | ...             | ...           | ...         | ...            | ...         | ...            | ...            | ...        |
| 1195 | ORD201195 | 2024-06-20 | C21126     | Desk    | 1        | 107.04    | 392 Main St     | Credit Card   | Cancelled   | TRK38009181    | 6           | FREESHIP       | Google         | 107.04     |
| 1196 | ORD201196 | 2024-03-04 | C20095     | Monitor | 2        | 662.53    | 778 Main St     | Online        | Cancelled   | TRK69207593    | 5           | No Coupon Code | Facebook       | 1325.06    |
| 1197 | ORD201197 | 2023-07-13 | C79674     | Tablet  | 2        | 436.84    | 275 Main St     | Online        | Delivered   | TRK88039356    | 2           | FREESHIP       | Instagram      | 873.68     |
| 1198 | ORD201198 | 2024-08-22 | C64753     | Chair   | 4        | 262.52    | 509 Main St     | Debit Card    | Cancelled   | TRK71683331    | 4           | WINTER15       | Instagram      | 1050.08    |
| 1199 | ORD201199 | 2023-06-11 | C57502     | Tablet  | 4        | 560.58    | 201 Main St     | Gift Card     | Returned    | TRK51116746    | 6           | SAVE10         | Referral       | 2242.32    |

1200 rows x 14 columns

### Step 2: Database Creation

A SQLite database was created using the SQLite3 library.

### Step 3: Data Import

The dataset was imported into a SQLite table for SQL analysis.

```
import sqlite3
conn=sqlite3.connect("Sales.db")
df.to_sql(name='sales',con=conn,if_exists='replace',index=False)
print("Dataset imported successfully")
```

[\*] Dataset imported successfully

Activate Windows  
Go to Settings to activate Windows.

## Step 4: Query Execution

SQL queries were executed to retrieve information from the database.

```
query="""SELECT * FROM sales LIMIT 5"""
result=pd.read_sql(query,conn)
result
```

[\*]

|   | OrderID   | Date                | CustomerID | Product | Quantity | UnitPrice | ShippingAddress | PaymentMethod | OrderStatus | TrackingNumber | ItemsInCart | CouponCode | ReferralSource | TotalPrice |
|---|-----------|---------------------|------------|---------|----------|-----------|-----------------|---------------|-------------|----------------|-------------|------------|----------------|------------|
| 0 | ORD200000 | 2023-01-04 00:00:00 | C72649     | Monitor | 5        | 570.62    | 928 Main St     | Debit Card    | Shipped     | TRK37947903    | 7           | SAVE10     | Instagram      | 2853.10    |
| 1 | ORD200001 | 2024-08-23 00:00:00 | C75739     | Phone   | 2        | 151.35    | 823 Main St     | Online        | Shipped     | TRK91186779    | 3           | SAVE10     | Referral       | 302.70     |
| 2 | ORD200002 | 2024-02-27 00:00:00 | C81728     | Tablet  | 5        | 550.68    | 512 Main St     | Credit Card   | Cancelled   | TRK42903982    | 8           | FREESHIP   | Email          | 2753.40    |
| 3 | ORD200003 | 2023-10-15 00:00:00 | C33540     | Chair   | 1        | 273.19    | 275 Main St     | Debit Card    | Returned    | TRK62788070    | 5           | SAVE10     | Facebook       | 273.19     |
| 4 | ORD200004 | 2025-05-08 00:00:00 | C81840     | Printer | 4        | 626.01    | 668 Main St     | Online        | Delivered   | TRK29241424    | 8           | SAVE10     | Email          | 2504.04    |

## Step 5: Data Filtering

WHERE clauses were used to filter records based on specific conditions.

```
query1="""
SELECT CustomerID,OrderStatus
FROM sales
WHERE PaymentMethod='Online'
"""
result1=pd.read_sql(query1,conn)
result1
```

[\*]

|     | CustomerID | OrderStatus |
|-----|------------|-------------|
| 0   | C75739     | Shipped     |
| 1   | C81840     | Delivered   |
| 2   | C93861     | Returned    |
| 3   | C34416     | Pending     |
| 4   | C17470     | Delivered   |
| ... | ...        | ...         |
| 253 | C82921     | Shipped     |
| 254 | C70246     | Pending     |
| 255 | C72007     | Pending     |
| 256 | C20095     | Cancelled   |
| 257 | C79674     | Delivered   |

258 rows x 2 columns

## Step 6: Data Grouping

GROUP BY clauses were used to categorize records and generate summarized results.

```
query6="""
SELECT Product,SUM(Quantity) AS Total_Quantity
FROM sales
GROUP BY Product
"""
result6=pd.read_sql(query6,conn)
result6
```

[\*] 0.0s

|   | Product | Total_Quantity |
|---|---------|----------------|
| 0 | Chair   | 562            |
| 1 | Desk    | 508            |
| 2 | Laptop  | 535            |
| 3 | Monitor | 480            |
| 4 | Phone   | 411            |
| 5 | Printer | 542            |
| 6 | Tablet  | 497            |

## Step 7: Aggregation Analysis

Aggregate functions were applied to calculate important metrics, including:

- COUNT()
- SUM()
- AVG()

```
query3="""
SELECT SUM(TotalPrice) AS Total_Revenue
FROM sales
"""
result3=pd.read_sql(query3,conn)
result3
```

| Total_Revenue |
|---------------|
| 0 1264761.96  |

```
query4="""
SELECT AVG(TotalPrice) AS Average_Revenue
FROM sales
"""
result4=pd.read_sql(query4,conn)
result4
```

| Average_Revenue |
|-----------------|
| 0 1053.9683     |

```
query5="""
SELECT COUNT(*) as Total_orders
FROM sales
"""
result5=pd.read_sql(query5,conn)
result5
```

| Total_orders |
|--------------|
| 0 1200       |

## Step 8: Result Sorting

ORDER BY clauses were used to sort query results in ascending or descending order.

```
query2 = """
SELECT OrderID, Product, TotalPrice
FROM sales
ORDER BY TotalPrice
"""
result2 = pd.read_sql(query2, conn)
result2
```

|      | OrderID   | Product | TotalPrice |
|------|-----------|---------|------------|
| 0    | ORD201161 | Phone   | 11.39      |
| 1    | ORD200863 | Phone   | 14.06      |
| 2    | ORD200240 | Tablet  | 17.24      |
| 3    | ORD200542 | Tablet  | 17.98      |
| 4    | ORD200336 | Laptop  | 18.20      |
| ...  | ...       | ...     | ...        |
| 1195 | ORD200328 | Tablet  | 3370.20    |
| 1196 | ORD200469 | Chair   | 3384.90    |
| 1197 | ORD200632 | Laptop  | 3390.80    |
| 1198 | ORD201122 | Monitor | 3390.95    |
| 1199 | ORD200789 | Tablet  | 3456.40    |

1200 rows x 3 columns

## Step 9: Interpretation

The results obtained from SQL queries were analyzed and interpreted to derive meaningful insights.

## 7. SQL Operations Performed

### Data Retrieval

- Retrieved all records using SELECT statements.
- Retrieved specific columns for analysis.

## Data Filtering

- Applied WHERE conditions to extract relevant records.
- Filtered data based on user-defined criteria.

## Data Grouping

- Grouped records using GROUP BY.
- Generated category-wise summaries.

## Aggregate Functions

- Used COUNT() to determine the number of records.
- Used SUM() to calculate total values.
- Used AVG() to calculate average values.

## Data Sorting

- Sorted results using ORDER BY.
- Displayed records in ascending and descending order.

## 8. Results

The SQL Data Analysis process successfully generated meaningful information from the dataset.

The analysis resulted in:

- Efficient retrieval of records from the database.
- Accurate filtering of relevant data.
- Group-wise summaries using aggregation functions.
- Statistical calculations using SQL functions.
- Improved understanding of the dataset through structured queries.

## 9. Key Observations

- SQL simplifies the process of data retrieval and analysis.
- SQLite3 provides an easy-to-use database environment within Jupyter Notebook.
- Aggregate functions help summarize large datasets efficiently.
- Grouping operations provide meaningful category-wise insights.
- Filtering operations enable focused analysis on specific subsets of data.

## 10. Key Skills Gained

- SQLite Database Management
- SQL Query Writing
- Data Import and Export
- Data Filtering using WHERE Clause
- Data Grouping using GROUP BY Clause
- Aggregate Functions (COUNT, SUM, AVG)
- Result Sorting using ORDER BY Clause
- Data Analysis and Interpretation

## 11. Challenges Faced

- Understanding SQL syntax and query structure.
- Importing data into the SQLite database.
- Writing accurate filtering conditions.
- Interpreting aggregated results.
- Managing large query outputs effectively.

## 12. Conclusion

The SQL Data Analysis project successfully demonstrated how SQLite3 and SQL can be used to analyze structured data efficiently. Various SQL operations such as SELECT, WHERE, GROUP BY, ORDER BY, COUNT(), SUM(), and AVG() were applied to generate meaningful insights from the dataset. The project enhanced understanding of database management and SQL-based data analysis techniques.

## 13. References

1. SQLite3 Documentation
2. SQL Documentation
3. Pandas Documentation
4. Jupyter Notebook Documentation
5. DecodeLabs Industrial Training Kit
6. Data Analytics Course Materials